



Savitribai Phule Pune University
Centre For Distance and Online Education

SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE

CENTRE FOR DISTANCE AND ONLINE EDUCATION

Program	Master of Computer Application	
Course	Programming from First Principles	
Topic Name	Values are to types as types are to classes (Practical Session) Part - 2	
Topic Code	-	
Unit/Module No. & Name	14	Type Classes
Self-Learning Hours	-	

Topic Details

S.No	Topic	Pg.No
1.0	Introduction	4
2.0	Meaning And Definition	5-7
3.0	Nature Or Type	8-10
4.0	Examples And Case Studies	11-13
5.0	Keywords And Touch Vocabulary	14



Savitribai Phule Pune University
Centre For Distance and Online Education

OBJECTIVE

1. The goal of this paper is to explain an important idea used in many programming courses.
2. This idea is that values relate to types, and types relate to classes.
3. The sentence “values are to types as types are to classes” is an analogy.
4. An analogy compares two relationships.
5. It shows that one pair of ideas works in a similar way to another pair.
6. In this analogy, the first pair is value and type.
7. The second pair is type and class.
8. The attached source is an audio file.
9. Because there is no transcript, I summarize the common concept taught in this topic.
10. By the end of this paper, you should be able to complete four tasks.
11. First, you should be able to define value, type, and class in clear language.
12. Second, you should be able to explain how a value belongs to a type.
13. You should also be able to explain how a type is supported by a class in many languages.
14. Third, you should be able to compare how Python, Java, and JavaScript express these ideas.
15. Fourth, you should be able to use the analogy when you design programs.
16. You should also be able to use it when you debug errors.
17. You should be able to use it when you read code written by others.

1.0 INTRODUCTION

Computers follow instructions exactly. A program is a set of instructions written in a programming language. When a program runs, it stores data, changes data, and produces output. To store data safely, the computer must know how large the data is, where it should be placed in memory, and which operations make sense for it. For example, a computer can add two numbers, but it cannot add a number to a color in a meaningful way. In the same way, a language can join two strings, but it usually cannot join a number and a sentence unless there is a clear rule. When we ignore these limits, we get bugs or confusing results.

Because of this, programming languages use categories for data. These categories are called types. Types guide the compiler or interpreter, and they also guide programmers. If you say a function returns a boolean, other people know it returns true or false. In many modern languages, especially object-oriented languages, classes are closely linked to types. A class is a design that describes data and behavior together. When you create an object from a class, the object value usually has that class as its type.

This topic matters for writing correct programs and designing good code. It also prepares you for advanced topics such as inheritance, interfaces, and polymorphism.



Savitribai Phule Pune University
Centre For Distance and Online Education

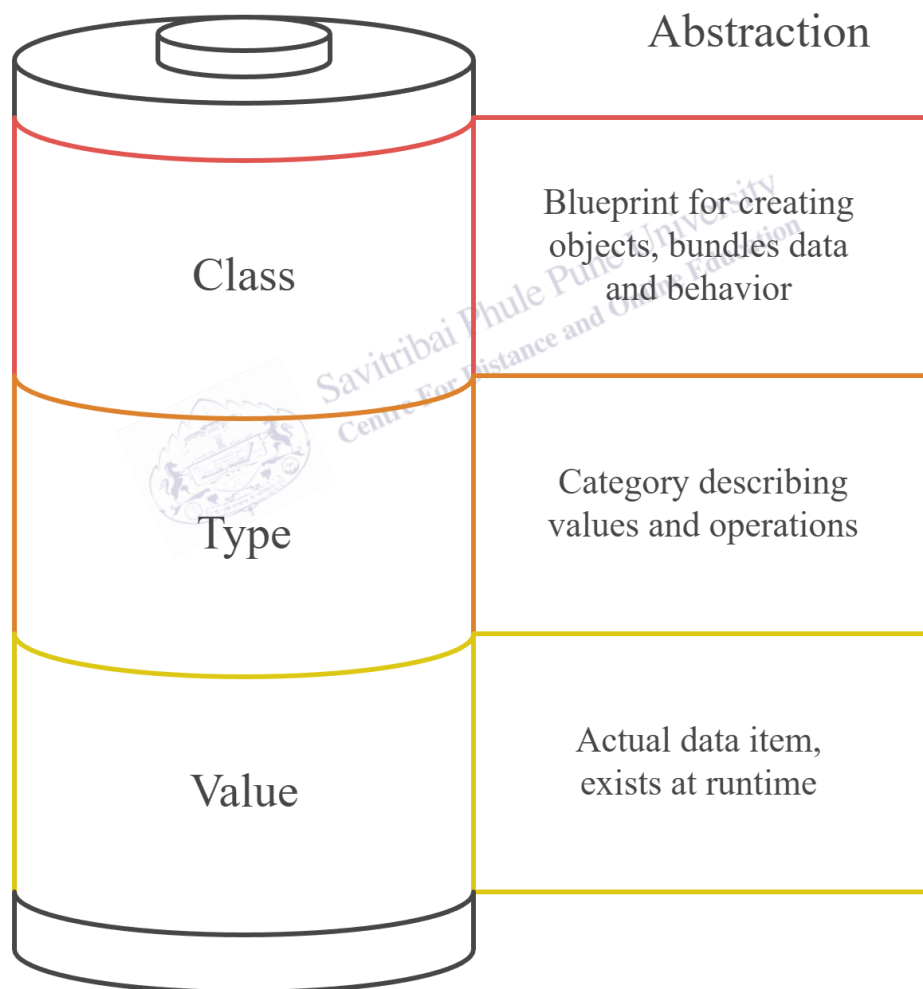
2.0 MEANING AND DEFINITION

2.1 Value

A value is a specific piece of data that a program can use. Values exist when the program is running. A value can be small, like the number 7, or complex, like a list of students, a map from names to scores, or an image. Values can be stored in memory, passed into functions, returned from functions, printed, or saved to files.

Fig. 1

Understanding data abstraction from concrete values to abstract blueprints



The diagram shows data abstraction moving from values to types and then to classes. A value is actual data at runtime. A type groups values and operations. A class is a blueprint that bundles data and behavior together for learners.

Values can be literal, meaning they are written directly in code, such as 10 or "hello". Values can also be computed, such as $2 + 3$, which becomes the value 5. Some values are immutable, meaning the value itself does not change. If you “change” an immutable value, you actually create a new value. Other values are mutable, meaning their internal parts can be updated, such as many lists or objects.

2.1.1 Key Features Of Values

Values have features that help us think clearly about programs. First, values are concrete. They are the real data items that operations act on. Second, some values have identity. Two objects can store the same text but still be different objects in memory. Third, values have representation. A number is stored in binary form. A string is stored as a sequence of character codes. These details matter when we study performance and memory.

2.2 Type

A type is a category that describes values and the operations that are valid for those values. A type answers two questions: what kinds of values are in this group, and what actions are allowed on those values? For example, the integer type includes whole numbers and allows addition and comparison. The string type includes text and allows joining, searching, and slicing. The boolean type includes true and false and allows logical actions such as AND and OR.

Types can be checked at different times. In statically typed languages, types are often checked before the program runs, usually by a compiler. In dynamically typed languages, types are checked while the program runs, usually by an interpreter. Both styles use types; the main difference is when the checks happen.

2.2.1 Type As A Rule Book

It helps to think of a type as a rule book. The rule book says which operations are meaningful. If you try to break the rules, you get an error. For example, dividing a string by a string is not listed as a valid string operation, so the language reports a type error. In a statically typed language, the compiler may stop you before you run the code. In a dynamically typed language, the error appears when the program reaches the wrong line.

This rule book idea also explains prediction. If you know a variable is an integer, you can expect that adding 1 gives another integer. If you know a value is a Student object, you can expect it has the methods defined by the Student class.

2.2.2 Type Conversion And Casting

Sometimes we need to move a value from one type to another. This process is called conversion. In some languages, conversion happens automatically in limited cases. In other languages, the programmer must request it. A simple example is turning the string "42" into the integer 42. The value changes because the representation changes, and the allowed operations change. After conversion, you can add 1 and get 43 as a number, not "421" as text.

In statically typed languages, conversion is often called casting. A cast tells the compiler how to treat a value. Some casts are safe, like converting an int to a long. Some casts are unsafe, like treating a Person as a Student when the value is not really a Student. In dynamically typed languages, conversion can still happen, but the checks happen at run time. Understanding conversion helps you see the boundary between values and types: you are not changing the variable name, you are changing the value and its category.

2.3 Class

A class is a blueprint or plan used to create objects. An object is a value that bundles data with operations that work on that data. In most object-oriented languages, a class defines fields and methods. Fields store data inside each object. Methods describe actions an object can perform. For example, a Student class might define fields such as name, id, and credits. It might define methods such as addCourse, computeGPA, or printTranscript.

Classes help organize large programs. A class groups related information and behavior. This grouping supports modular design, reuse, and clearer reading. It also supports abstraction, because users of a class can focus on what the object does, not on how it is stored internally.

2.3.1 Object And Instance

When you create an object from a class, the object is called an instance of the class. The class is the plan, and the instance is the real value created from the plan. Two instances made from the same class share the same structure and method names, but they can store different field values. For example, two Student objects may both have name and id fields, but one may store “Asha” and the other may store “Ravi.”

Each instance is a separate value in memory. Even if two instances look similar, they are not the same object unless they refer to the same memory location. This difference matters when you copy or update objects.

2.4 The Analogy Explained

The analogy connects three levels. At the lowest level, we have values, which are the actual data items. At the next level, we have types, which are categories for values. Saying “a value has a type” means the value fits into a category with known rules. At a higher level, we often have classes, which are structured definitions that describe how certain kinds of objects behave. In many languages, a class defines a type for its instances. That is why we say “types relate to classes.”

The analogy is not perfect in every language. Some languages have types that are not classes, like primitive types in Java. Some languages describe types without classes, like interface or shape types. Still, the analogy is useful. It reminds students to ask: what is the value, what type rules apply, and what definition provides the behavior?



3.0 NATURE OR TYPE

3.1 Static And Dynamic Typing

Typing can be described as static or dynamic. In static typing, the type of a variable or expression is checked before the program runs. This checking often happens during compilation. The compiler can catch mistakes early, such as passing the wrong kind of argument to a function. Java and C# are common examples.

Fig. 2

Programming Language Typing Systems

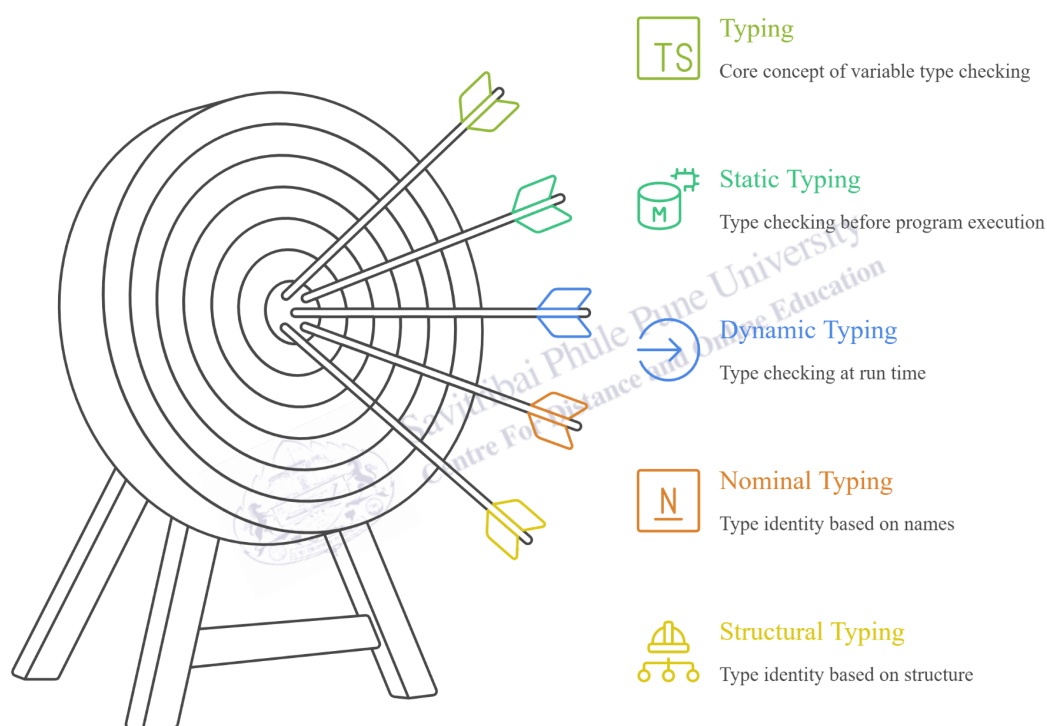


Diagram explains programming language typing systems. Typing checks variable types. Static typing checks before execution. Dynamic typing checks at runtime. Nominal typing depends on names. Structural typing depends on structure. Arrows hitting targets show different approaches in computer science education.

In dynamic typing, the type is checked at run time. The program can be more flexible because variables can be bound to values of different types as the program runs. Python and JavaScript are common examples. Many languages now mix ideas. TypeScript adds static checks to JavaScript. Python supports optional type hints that tools can analyze.

3.1.1 Variables And Binding

A variable is a name that refers to a value. The act of linking a name to a value is called binding. In many dynamic languages, the same name can be bound to different types over time. In many static languages, the type of the variable is fixed after declaration, so rebinding to a different type is not allowed without conversion.

Understanding binding helps students avoid confusion between a name and a value. The name is not the value. When a name changes its binding, the old value may still exist if other names point to it, or it may be removed by garbage collection if nothing refers to it.

3.2 Primitive Types And Class Types

Many languages include primitive types. Primitive types are built-in types designed to be efficient and simple, such as int or boolean. In Java, int is a primitive type and is not a class. Java also has wrapper classes, such as Integer and Boolean. Wrapper objects can store primitive values and provide methods. They also allow primitives to be used in data structures that store objects.

In Python, many values that look primitive, such as integers, are still objects. The type int is also a class, and integer values have methods. In JavaScript, numbers behave like primitives in many cases, but the language can temporarily wrap them in objects to provide methods. These differences show that languages choose different designs. Still, the basic relationships remain: values belong to types, and when values are objects, their behavior is often defined by a class or class-like structure.

3.3 Nominal And Structural Typing

Types can also be described by how the language decides type identity. In nominal typing, type identity depends on names. If two classes have different names, the language treats them as different types, even if they have the same fields. Java is mostly nominal. In structural typing, type identity depends on structure. If a value has the required fields and methods, it can match the type, even if it comes from a different class name. TypeScript uses structural typing in many cases.

In nominal systems, classes strongly control types because the class name is part of the type. In structural systems, types can be described without a single class, because a type can be a “shape” that many classes can satisfy. Even so, classes still matter because classes create values with predictable structure.

3.4 Type Hierarchies And Inheritance

In object-oriented programming, classes can be related through inheritance. Inheritance means one class can extend another class, reusing and improving its design. For example, a Person class might define name and age. A Student class might extend Person and add studentId and credits. In many languages, this also creates a type hierarchy. A Student value

can often be used where a Person value is expected, because Student is a kind of Person. This idea is called substitutability.

Type hierarchies connect to the analogy because they show that types can be organized. Values are still the concrete items. Types still group values. Classes still provide the blueprint that creates values and positions them in the type system. When you understand the hierarchy, you can reason about which methods are available and which assignments are safe. You can also understand polymorphism, where the same call can behave differently depending on the actual class of the value.

3.5 Types In Memory And References

A final point is how values live in memory. Some languages store small values directly, while other values are stored as objects with references. A reference is like an address that points to where the value is stored. If two variables hold the same reference, they point to the same object value. Then a change through one variable can be seen through the other. This is common with lists and other mutable objects.

Types and classes help manage this memory story. If a value is an object, its class tells the runtime system how big the object is, how to find its fields, and which methods can be called. The key lesson is that a value can be direct data or an object reached through a reference, but it still has a type.



4.0 EXAMPLES AND CASE STUDIES

4.1 Example In Python

Consider Python code: `x = 5`. Here, 5 is a value. Its type is `int`. In Python, `int` is also a class because Python treats many types as classes. You can call methods such as `bit_length` on the integer value. If you later write `x = "5"`, the new value is the string "5" and its type is `str`. The name `x` is not fixed to one type. It is bound to whichever value you assign at that time.

This example shows the analogy directly. The value 5 belongs to the type `int`. The type `int` is also a class that defines behavior shared by all integer values. If you create a custom class, such as `Student`, then `Student` is a class, and it also creates a type for its instances. A `Student` instance is a value, and its type is `Student`.

4.1.1 Example In Java

Now consider Java code: `int x = 5`; Here, 5 is a value. Its type is `int`. The variable `x` is declared as `int`, and it cannot later store a string value. This is a key feature of static typing: the variable's type controls what can be assigned to it.

Java also shows the boundary between types and classes. The type `int` is not a class, but Java provides the class `Integer`. If you write `Integer y = 5`;, Java uses autoboxing to turn the `int` value into an `Integer` object. Then `y` refers to an object value, its type is `Integer`, and `Integer` is a class. In this way, many object types in Java are tied to classes, while primitive types are not.

4.1.2 Example In JavaScript

JavaScript is dynamically typed. If you write `let x = 5`;, then 5 is a value and its type is `number`. Later, `x` can be assigned "five", a value with type `string`. JavaScript shares object behavior through prototypes, and modern syntax can look like classes. When a value is an object, it behaves according to the structure that created it.

4.2 Case Study: Debugging A Type Error

Imagine a small program that reads two inputs and adds them. In Python, keyboard input is often a string. If you forget to convert the inputs, you might add "3" and "4". In Python, "3" + "4" produces the string value "34", not the number 7. If you expected 7, this is a logic error. If you try `3 + "4"`, Python raises a type error because adding an `int` and a `str` is not defined.

This case study shows that types protect meaning. The values "3" and "4" are valid, but they belong to the string type, so the plus operator follows string rules, not numeric rules. The fix is to convert the values into integers, so they belong to the `int` type. In Java, the compiler supports this by requiring clearer conversions. In JavaScript, automatic conversion can hide errors, so explicit checks are often needed.

4.2.1 Case Study: Modeling Students With A Class

Suppose you are building a simple university system. You need to store student data and allow actions such as enrolling in courses. You can define a Student class with fields such as name, id, credits, and courses. You can define methods such as addCourse, dropCourse, and calculateFees. Each Student object created from the class is a value. The type of that value is Student, and the Student class defines what the value can do.

This case study shows how classes add meaning. Without a class, you might store student data in separate lists or dictionaries. That can work, but it can be harder to keep the data consistent. A class keeps the data and behavior in one place and creates a clear type. If a function expects a Student, you should not pass a Course or a number.

4.3 Case Study: Data Structures And Generic Types

Data structures are a core topic in undergraduate computer science. Consider a list that stores grades. In Java, you might use List<Integer>. This type says the list should store Integer values, not strings or Student objects. The list itself may be implemented by a class such as ArrayList, and the element type is part of the full type. When you try to add a value of the wrong type, the compiler warns you early.

In Python, lists can store mixed types. That flexibility is useful for quick scripts, but it can create confusion in larger systems. Python supports type hints, so you can write list[int] to show that you intend to store integers. Tools can check this and warn you. TypeScript uses generics in a similar way. Across languages, the pattern is the same: values are stored in containers, types describe which values should be allowed, and classes often implement the container behavior.

4.4 Case Study: Interfaces And Expected Behavior

In many languages, a type can be defined by an interface, which is a list of required methods. For example, a Printable interface might require a print method. A class can implement that interface by providing the method. Then values made from that class can be used where the interface type is expected. This approach is common in Java and C#.

This case study deepens the analogy. The value is still the concrete object. The type can be an interface, which describes expected behavior. The class provides the real method code. So, the relationship “type to class” can mean that a type describes behavior, while a class supplies an implementation. This helps teamwork because developers agree on interface types and then build different classes that meet those types.

4.5 Case Study: Reading Function Signatures

In undergraduate courses, you often read library code. Many libraries show function signatures, such as sort(list[int]) -> list[int]. This signature is a short statement about types. It says the input should be a list of integer values, and the output will also be a list of integer values. The signature does not show exact values; it describes types.

Now connect the analogy. When you call the function, you pass values. The function checks or assumes the types. If the values do not match the expected types, the program may fail or return the wrong result. If the elements are objects, their classes define the methods that sort might use, such as comparison. Often, a class provides the needed behavior.

1. Values are the real data items used during execution.
2. Types group values and define valid operations.
3. Classes often define object types by describing fields and methods.
4. Variables bind names to values, and bindings can change in some languages.
5. Static typing checks early; dynamic typing checks at run time.
6. Wrapper classes connect primitive data to object behavior.
7. Inheritance builds type hierarchies and supports substitutability.
8. Generics restrict the kinds of values allowed in collections.



5.0 Keyword Touch Vocabulary Meaning

Keyword	Touch Vocabulary (Simple Meaning)
Value	A specific piece of data, like 5 or "cat"
Type	A category that describes allowed operations
Class	A blueprint for creating objects
Object	A value with data and methods
Instance	One object created from a class
Variable	A name that refers to a value
Binding	Linking a name to a value
Static typing	Checking types before running code
Dynamic typing	Checking types while code runs
Primitive	Basic built-in type like int
Wrapper	Object form of a primitive
Method	Action defined for an object
Field	Data stored inside an object
Interface	Type defined by required methods
Inheritance	Extending a class to reuse design
Hierarchy	Parent-child ordering of types
Generic	Type that takes another type
Nominal typing	Type identity based on names
Structural typing	Type identity based on structure
Prototype	Shared behavior source in JavaScript